Final Project Report

1. Project Overview

This project focuses on both automation testing using Selenium WebDriver with Node.js and manual testing, including preparing reports for test cases and bugs. The main objective was to automate basic test cases such as login validation and dashboard functionality.

For automation testing, a demo website (SauceDemo) was used because it is stable and suitable for practice. Manual testing focused on the Facebook login page, where test cases were created and bug reports were prepared based on the observed issues.

---

2. Testing Methodology

This project used a combination of automation testing and manual testing.

- Automation Testing: Selenium WebDriver was used to control the browser and perform actions such as opening websites, entering data, clicking buttons, and verifying results. Explicit waits were implemented to handle dynamic elements and ensure they loaded before interaction. Automation scripts were executed using Node.js.
- Manual Testing: Manual test cases were written for the Facebook login page. Bugs were identified based on incorrect behavior, missing validations, or inconsistent error messages.

---

3. Test Case Summary

Test Case 1: Empty Login

- Steps:
    1. Open website
    2. Leave username and password empty
    3. Click login button
- Expected Result: Error message should be displayed
- Actual Result: Error message displayed
- Status: PASS

Test Case 2: Invalid Login

- Steps:
    1. Enter incorrect username and password
    2. Click login button
- Expected Result: Login should fail and show error
- Actual Result: Error message displayed
- Status: PASS

Test Case 3: Dashboard Functionality

- Steps:
    1. Enter valid username and password
    2. Click login button
- Expected Result: Dashboard page should open
- Actual Result: Products page loaded successfully
- Status: PASS

Facebook Login Page Test Summary

Site: Meta Platforms (Facebook)
Feature under test: Login Page

Total Test Cases: 15

Passed Test Cases

- FB-01: Login with valid credentials → Login successful, homepage opened
- FB-02: Login with invalid password → Correct error message displayed
- FB-03: Login with invalid email → Correct error message displayed
- FB-05: Password masking → Password hidden as expected
- FB-08: Login button functionality → Works correctly
- FB-09: Forgot Password link → Opens recovery page successfully

- FB-11: Email field input → Text entered successfully
- FB-12 (multiple wrong attempts): No security alerts triggered (default behavior)
- FB-12 (Create New Account): Registration form opened successfully
- FB-13: Design verification → Matches Figma UI

Failed Test Cases / Issues Found

- FB-04: Empty fields → Generic error message shown; should prompt "Required email and password"
- FB-06: Only email entered → Incorrect password message shown; field-specific validation missing
- FB-07: Only password entered → Email validation missing; generic error displayed
- FB-10: Spaces in email field → Validation missing; spaces allowed
- FB-14: Very long email → No restriction; input accepted but login fails
- FB-15: Very long password → No restriction; input accepted but login fails

Summary

- Main issues: Input validation for empty fields, long inputs, and partial fields is inconsistent.
- UI/UX: Design matches expected Figma layout. Password masking works correctly.
- Functional: Login button, Forgot Password, and Create New Account links work as expected.
- Security/Validation: Multiple rapid attempts and long inputs are not fully validated.

---

4. Bug Report Summary

Seven bugs were identified on the Facebook login page.
Most issues were related to input validation, such as empty fields, long inputs, and spaces in the email field.
Some UI behaviors, like rapid clicking of the login button, were also not handled

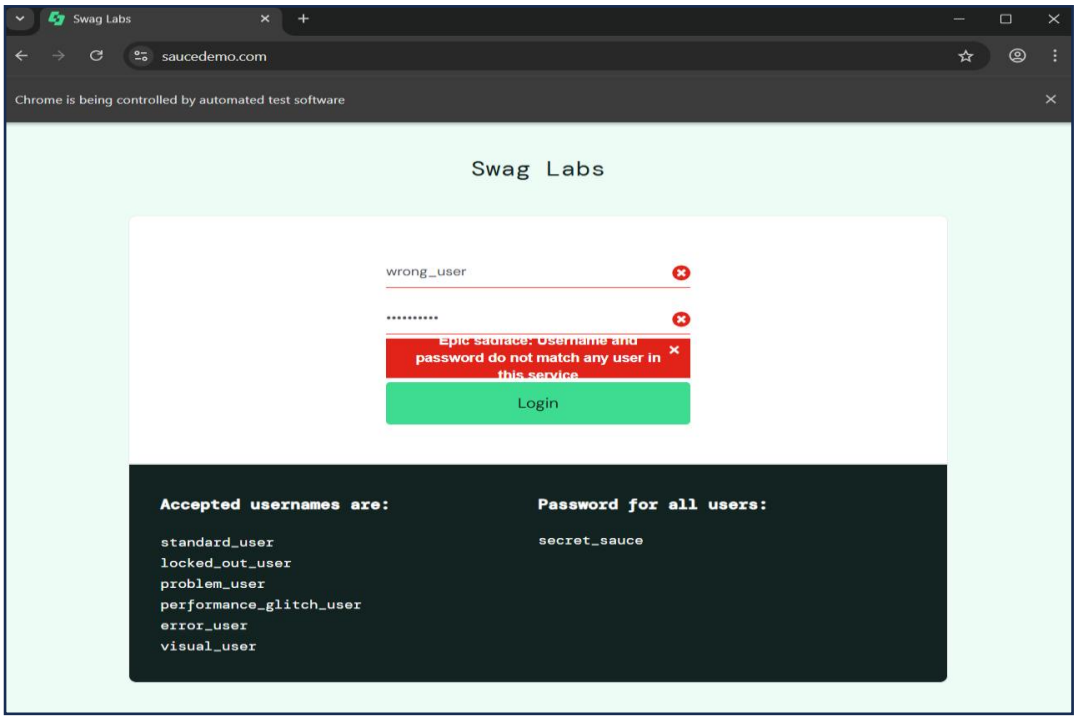
properly.
All issues affect user experience but do not completely block login functionality.

---

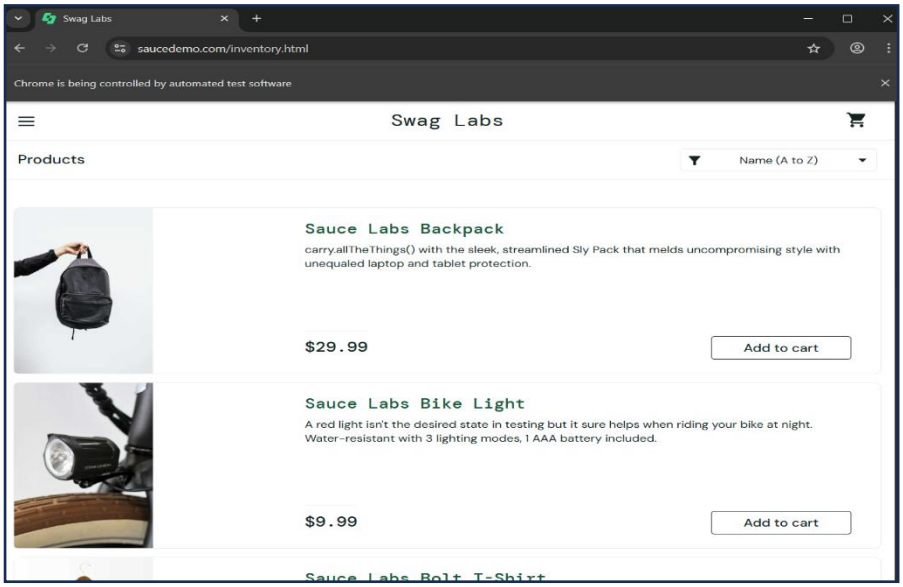
5. Automation Scripts (Code) (GitHub)

Mention in (GitHub)

Login Test Script



Dashboard Test Script



---

6. Key Learnings

- Learned to use Selenium WebDriver with Node.js for browser automation
- Learned how to implement explicit waits for dynamic elements
- Understood automation of login and dashboard functionality
- Learned to validate test results and log PASS/FAIL statuses
- Learned the importance of input validation and error handling in UI

---

7. Recommendations

- Always use test-friendly websites for automation practice
- Implement explicit waits instead of fixed delays for stable testing
- Avoid using real user accounts for testing
- Keep automation scripts simple, readable, and modular
- Use proper validation for input fields and error messages
- Improve security measures for multiple failed login attempts

---

Conclusion

The project successfully demonstrated the use of automation and manual testing. Automated scripts for login and dashboard functionality were implemented and executed successfully. Manual testing of the Facebook login page highlighted several bugs related to input validation and error messaging. Overall, the project improved understanding of automation testing, bug reporting, and test case documentation.